

Documentación técnica y operativa

1. Resumen

Osorio Property Hub es un portal inmobiliario con dos áreas:

- **Público:** muestra el sitio (home, búsqueda, listado, detalle, calculadora, mapa) utilizando datos mock
- **Admin:** permite administrar el contenido usando **Supabase** (Auth + CRUD + Storage).

Durante este proyecto, la integración a base de datos está diseñada para operar **multi-tenant** por `store_id` (un `store_id` por empresa).

2. Stack tecnológico

- **Frontend:** Vite + React 18 + TypeScript
- **Ruteo:** `react-router-dom`
- **Estilo:** Tailwind CSS + componentes estilo `shadcn/ui` (basados en Radix UI)
- **Estado global:**
 - `LanguageContext` para i18n (es, en, pt)
 - `AuthContext` para sesión y perfil admin
- **UI:** `lucide-react` (iconos), `sonner` (toasts)
- **Mapa:** `react-leaflet` + `leaflet`
- **Supabase:** `@supabase/supabase-js`

3. Arquitectura y ruteo

El ruteo está centralizado en `src/App.tsx`.

3.1. Rutas públicas

- `/` -> `src/pages/Index.tsx`
- `/propiedades` -> `src/pages/Properties.tsx`
- `/propiedad/:id` -> `src/pages/PropertyDetail.tsx`

3.2. Rutas admin

- `/admin/login` -> `src/pages/admin/AdminLogin.tsx`
- `/admin/dashboard` -> `src/pages/admin/AdminDashboardPage.tsx` (protegida)
- `/admin/properties` -> `src/pages/admin/AdminPropertiesPage.tsx` (protegida)
- `/admin/properties/new` -> `src/pages/admin/AdminPropertyFormPage.tsx` (protegida)
- `/admin/properties/:id/edit` -> `src/pages/admin/AdminPropertyFormPage.tsx` (protegida)
- `/admin/quotes` -> `src/pages/admin/AdminQuotesPage.tsx` (protegida)
- `/admin/neighborhoods` -> `src/pages/admin/AdminNeighborhoodsPage.tsx` (protegida)

La protección se hace con:

- `src/components/admin/AdminProtectedRoute.tsx`

4. Internacionalización (i18n)

- Idiomas soportados: es, en, pt
- Traducciones en: `src/i18n/translations.ts`
- Conmutación de idioma desde: `src/components/layout/Navbar.tsx`

Uso típico en UI:

- `const { t } = useLanguage();`
- `Render: {t('hero.title')}, {t('properties.title')}, etc.`

5. Integración Supabase (Auth y datos)

5.1. Cliente Supabase

Creación en: `src/lib/supabase.ts`

5.2. Auth y perfil admin

Lógica en: `src/contexts/AuthContext.tsx`

- `supabase.auth.getSession()` al montar
- `supabase.auth.onAuthStateChange()` para escuchar cambios de sesión
- Carga de perfil:
 - `tabla: profiles`
 - `filtro: .eq('id', session.user.id).single()`

El perfil se usa para:

- comprobar `profile.role === 'admin'`
- aplicar `store_id` en consultas multi-tenant

6. Flujo del sitio público (mock)

Nota: actualmente el sitio público utiliza `MOCK_PROPERTIES`, `MOCK_NEIGHBORHOODS`, etc. en vez de Supabase.

6.1. Home / (composición)

Composición en: `src/pages/Index.tsx`

- `HeroSection`
- `FeaturedProperties`
- `TransitionBanner`
- `MapPreview`
- `CalculatorSection`
- `TestimonialsSection`

6.1.1 Hero + búsqueda

Archivo: `src/components/home/HeroSection.tsx`

- Muestra `t('hero.title')` y `t('hero.subtitle')`
- Permite seleccionar:
 - barrio
 - tipo
 - operación
- Permite búsqueda por texto
- Navega a `/propiedades` con `query string`:
 - `barrio, tipo, operacion, q`

6.2. Listado `/propiedades`

Archivo: `src/pages/Properties.tsx`

- Lee query params:
 - `barrio, tipo, operacion, q`
- Filtra y ordena en cliente sobre `MOCK_PROPERTIES`
- Sorting:
 - `quotes: quote_count desc`
 - `price_asc / price_desc`
 - `newest` (en mock no modifica)

6.3. Detalle `/propiedad/:id`

Archivo: `src/pages/PropertyDetail.tsx`

- Busca la propiedad en `MOCK_PROPERTIES` por `id`
- Si no existe: muestra pantalla “Propiedad no encontrada”
- Renderiza:
 - badges por estado y operacion
 - galería (mock principal + recursos fijos)
 - descripción y características
 - sidebar con botón “Solicitar cotización”

6.3.1 Quote modal

Archivo: `src/components/property/QuoteModal.tsx`

- Formulario: nombre, teléfono, email, mensaje
- Al enviar: **actualmente no guarda en Supabase** (contiene `TODO`)

6.4. Mapa

Archivo: `src/components/home/MapPreview.tsx`

- Muestra mapa centrado en Asunción
- Renderiza marcadores con propiedades mock
- Link "Ver detalle" a `/propiedad/:id`

6.5. Calculadora

Archivo: `src/components/home/CalculatorSection.tsx`

- Inputs:
 - barrio
 - tipo (selección UI)
 - área (m²)
- Cálculo:
 - `estimated = area * price_m2`
 - `min = estimated * min_factor`
 - `max = estimated * max_factor`

7. Flujo del panel Admin (Supabase real)

Nota: en el panel admin sí se consultan tablas de Supabase y se usa Storage.

7.1. Autenticación y navegación

- Login: `src/pages/admin/AdminLogin.tsx`
- Protección: `AdminProtectedRoute`
- Sidebar: `src/components/admin/AdminSidebar.tsx`

7.2. Dashboard `/admin/dashboard`

Archivo: `src/pages/admin/AdminDashboardPage.tsx`

- Usa `profile.store_id`
- Consultas (en paralelo):
 - `products_inmobiliaria`: métricas y top por `quote_count`
 - `quotes`: últimos 20 por `created_at` desc
- Render:
 - tarjetas de métricas
 - top propiedades
 - últimas cotizaciones

7.3. Propiedades (CRUD) `/admin/properties`

Archivo: `src/pages/admin/AdminPropertiesPage.tsx`

- Lectura:
 - `products_inmobiliaria` filtrado por `store_id`
 - `neighborhoods` filtrado por `store_id` para mostrar nombre
- Filtros (cliente):
 - búsqueda por título / barrio
 - status, tipo, operación
- Escritura:

- eliminar propiedad con `delete()` + filtros por `id` y `store_id`

7.3.1 Formulario (Create/Edit) `/admin/properties/new|:id/edit`

Archivo: `src/pages/admin/AdminPropertyFormPage.tsx`

- Carga dependencias:
 - `neighborhoods` por `store_id`
- Si `isEdit`:
 - fetch de la propiedad por `id` y `store_id`
- Submit:
 - validación básica
 - `insert()` o `update()` sobre `products_inmobiliaria` (con `store_id`)

7.4. Imágenes de propiedades (Storage + tabla)

Archivo: `src/components/admin/AdminPropertyImagesManager.tsx`

- Lectura:
 - `product_images_inmobiliaria` filtrado por `product_id` y `store_id`
- Subida:
 - Storage bucket: `property-images`
 - path: `${storeId}/${propertyId}/${timestamp}-${random}.${ext}`
 - se guarda URL pública en `product_images_inmobiliaria`
- Operaciones:
 - marcar `is_primary`
 - reorder por `sort_order`
 - eliminar por `id` + `store_id`

7.5. Cotizaciones (lectura y filtros) `/admin/quotes`

Archivo: `src/pages/admin/AdminQuotesPage.tsx`

- Lectura:
 - `quotes` filtrado por `store_id`
 - `products_inmobiliaria` para mapear `product_id` -> `title`
- Filtros en cliente:
 - por propiedad
 - por status

Importante: no se observa un update de status desde el panel.

7.6. Barrios (CRUD) `/admin/neighborhoods`

Archivo: `src/pages/admin/AdminNeighborhoodsPage.tsx`

- Lectura:
 - `neighborhoods` filtrado por `store_id`
- Create/Update:
 - inserta/actualiza campos y usa `store_id`
- Delete:
 - elimina por `id` + `store_id`

8. Requisito Supabase: usar siempre schema `Osorio_Inmuebles`

Ustedes definieron que **siempre** se debe usar el esquema `Osorio_Inmuebles` para crear todas las tablas.

En el código actual, las consultas se hacen como:

- `supabase.from('tabla')`

Esto típicamente consulta el **schema por defecto** (frecuentemente `public`) si no se especifica otro.

8.1. Recomendación de implementación (para cumplir el requisito)

En Supabase, con el cliente `supabase-js`, la forma habitual es:

- crear un “sub-client” por schema:
 - `const osorio = supabase.schema('Osorio_Inmuebles')`
- y reemplazar:
 - `supabase.from('tabla')` por `osorio.from('tabla')`

Esto debe aplicarse a:

- `profiles`
- `products_inmobiliaria`
- `neighborhoods`
- `quotes`
- `product_images_inmobiliaria`

8.2. Seguridad real

Aunque el frontend filtra por `store_id`, la seguridad debe garantizarse en Supabase con:

- políticas RLS por tabla
- políticas que validen `store_id`

9. Notas y limitaciones actuales

- Público usa **mock** (no hay lectura real desde Supabase).
- `QuoteModal` tiene un TODO y **no guarda** en quotes.
- En `npm run lint` existen errores/warnings preexistentes (relacionados a tipos `any` y estructura), aunque no son el foco del cambio de texto.

10. Cómo ejecutar localmente

- **Instalar:** `npm install`
- **Desarrollo:** `npm run dev` (servidor en el puerto configurado por `vite.config.ts`)
- **Lint:** `npm run lint`